



Práctica 2

Objetivo

El alumno se familiarizará con el uso de tareas usando el sistema embebido ESP32 DevKit v1 para desarrollar aplicaciones para sistemas basados en microcontrolador para aplicarlos en la resolución de problemas de cómputo, de una manera eficaz y responsable

Equipo

Computadora personal con conexión a Internet.

Teoría

- Describa a detalle la función `xTaskCreatePinnedToCore` para crea una tarea que esté anclada a un core en particular. Incluya un ejemplo de uso.
- Describa a detalle la función `xEventGroupSync` la cual activa bits dentro de un grupo de eventos y luego espera a que se active una combinación de bits dentro del mismo grupo de eventos. Incluya un ejemplo de uso diferente al de la documentación.

Desarrollo

Implemente en el ESP32 ESP-IDF el juego *Space Defender*, haciendo uso de **tareas y mecanismos de sincronización de tareas**. La implementación debe ser eficiente en el uso de recursos de cómputo (procesador, memoria y periféricos). La implementación se realiza en el **board ESP32**, no en el simulador.

Descripción del juego:

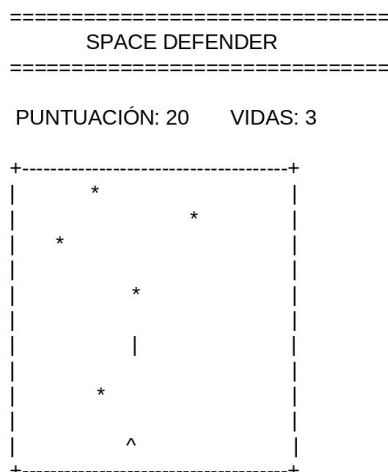


Fig. 1. Juego *Space Defender*.

- El jugador controla una nave espacial y debe defenderse de una oleada de enemigos. Máximo puede haber **5 enemigos simultáneos**.
- Cuando el juego inicia, el jugador tiene **3 vidas**.
- La terminal tiene un espacio rectangular en el que los enemigos aparecen en la parte superior y la nave está en la parte de abajo. Las posiciones en el **eje X** de los enemigos son **aleatorias**. Los enemigos se desplazan hacia abajo en línea recta hacia la nave del jugador.
- La nave se puede desplazar hacia la izquierda y derecha dentro del espacio rectangular, sin salirse de el.

- La nave dispara proyectiles en línea recta hacia los enemigos. Si el proyectil coincide con la posición de un enemigo, el enemigo y el proyectil del enemigo desaparecen y el jugador obtiene **1 punto**.
- Los enemigos disparan proyectiles en línea recta hacia la nave. El enemigo que dispara en un momento dado es elegido de forma aleatoria. Cada vez que un enemigo o un proyectil impacta contra la nave, el jugador **pierde una vida**. Cuando las vidas llegan a cero, el juego termina y el jugador pierde.
- Cuando un enemigo sale del tablero, desaparece y libera espacio para que pueda aparecer otro enemigo.
- El objetivo del jugador es **sobrevivir el mayor tiempo posible y obtener la mayor puntuación**, destruyendo los enemigos antes de que alcancen la nave.
- El juego muestra en LEDs el tiempo en segundos que el jugador ha acumulado en el juego. Y muestra en pantalla el puntaje y número de vidas que tiene.
- El juego termina cuando las vidas llegan a cero o cuando el jugador decide terminar.

Al final del juego, si el jugador decidió terminarlo, se despliega el mensaje “Hasta luego”. O si el jugador perdió, se despliega “Suerte para la proxima”. También, en ambos casos se despliega la puntuación que obtuvo el jugador y la puntuación máxima de todos los juegos que se han ejecutado desde que arrancó el ESP32.

Conecte tres botones al ESP32. Un botón se usa para desplazar la nave una posición a la izquierda cada vez que el jugador lo presiona. El segundo botón es similar pero para desplazar a la derecha. El tercer botón se usa para que la nave dispare un proyectil cuando el juego esta activo, y para iniciar un nuevo juego cuando el anterior terminó.

Conecte 5 LEDs al ESP32 y despliegue en binario el tiempo en segundos que el jugador ha acumulado en el juego.

El programa debe estructurarse en al menos cuatro tareas:

- **Tarea de entrada:** detecta los botones y genera eventos de movimiento o disparo.
- **Tarea del juego:** actualiza posiciones, colisiones, vidas y puntuación.
- **Tarea de enemigos:** genera periódicamente nuevos enemigos cuando se libera espacio para más enemigos.
- **Tarea de pantalla:** recibe el estado del juego y actualiza la terminal.

Puede agregar más tareas según lo considere necesario.

En el reporte incluya:

- Una descripción de las tareas que implementó y la función de cada tarea.
- Una descripción de los mecanismos de colas, Mutex, semáforos y Event groups que usó indicando por qué y para qué los uso.

Conclusiones y comentarios
Dificultades en el desarrollo
Referencias